

DL Complexity

Theoretically informed deep learning model complexity estimation

Alyabushev, Dementiev, Kurdyukov, Chirkov

Python package: `dl_complexity`

Technical Meeting 1

Problem statement

Parameter count alone neither determines the complexity of the represented function nor explains generalization in overparameterized networks.

Research question: which theoretically motivated estimators are most strongly associated with held-out NLL and the generalization gap across dataset sizes, architectures, and regularization settings?

- Implement several estimator families behind a common interface.
- Compare them under one controlled experimental protocol.
- Report the score, computational cost, rank stability, assumptions, and warnings.

The project compares useful approximations; it does not assume one universal true measure of neural-network complexity.

Project deliverables and scope

Deliverables

- an installable Python library;
- a common benchmark runner;
- reproducible configurations;
- comparison tables and plots;
- documentation and a demonstration workflow.

First release

- supervised classification;
- small MLPs and CNNs;
- synthetic data, MNIST, and Fashion-MNIST;
- PyTorch models;
- checkpoint-based evaluation where possible.

Estimator-specific work: posterior sampling, curvature estimation, compression, or repeated training is performed when the method requires it.

Out of scope: large language models, distributed training, automatic model selection, and proofs of new generalization bounds.

Current status: planning; the estimators have not yet been implemented.

Project name and common interface

Project name: DL Complexity.

Python package: `dl_complexity`.

Primary interface

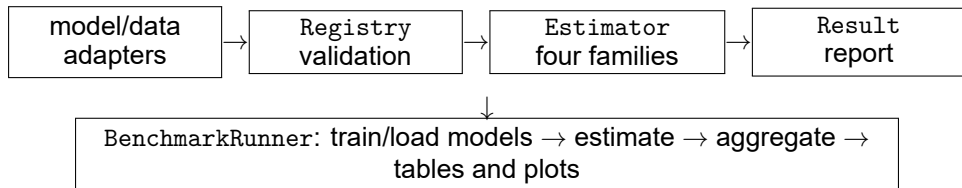
```
ComplexityEstimator.estimate(context) -> ComplexityResult
```

Type	Purpose
EstimationContext	model, dataset, split, likelihood, training artifacts, seed
ComplexityResult	native value, valid nats/bits conversion, per-sample value, target, direction components, assumptions, warnings, runtime, peak memory, metadata
EstimatorConfig	prior, precision, blocks, posterior samples, compression settings

With natural logarithms, AIC/BIC/HQIC/WAIC are in deviance units: divide by 2 for an NLL-like value in nats and by $2 \ln 2$ for bits. This conversion does not make them literal prefix-code lengths.

One output schema does not imply one semantics: predictive criteria, evidence approximations, operational codes, and risk bounds remain distinct.

Library architecture, integration, and technology stack



Integration

- `torch.nn.Module`, `DataLoader`;
- likelihood and posterior-sample adapters;
- optional curvature and compression backends;
- JSON/CSV-compatible result records.

Technology stack

- Python 3.11+, PyTorch;
- NumPy, SciPy, pandas, scikit-learn;
- pytest, coverage, ruff, static typing;
- MkDocs; Jupyter or Streamlit for the demo.

Algorithms I: information criteria and two-part MDL

Let $\hat{L}$ be the maximized *total* likelihood, k the number of estimated free parameters, and n the number of independent observations. Natural logs are used.

$$\text{AIC} = -2 \log \hat{L} + 2k, \quad \text{BIC} = -2 \log \hat{L} + k \log n, \quad \text{HQIC} = -2 \log \hat{L} + 2k \log \log n.$$

WAIC is a posterior predictive criterion:

$$\text{WAIC} = -2(\text{lppd} - p_{\text{WAIC}}), \quad \text{lppd} = \sum_i \log \mathbb{E}_{\theta|D} p(y_i | \theta), \quad p_{\text{WAIC}} = \sum_i \text{Var}_{\theta|D} \log p(y_i | \theta).$$

WBIC approximates Bayesian free energy using a tempered posterior:

$$p_\beta(\theta | D) \propto p(D | \theta)^\beta p(\theta), \quad \beta = 1/\log n, \quad \text{WBIC} = \mathbb{E}_\beta[-\log p(D | \theta)].$$

Two-part MDL is an operational code:

$$L_{2p} = L(\theta_\delta) + L(D | \theta_\delta),$$

where a prefix code, quantization rule δ , model structure, and hyperparameters must be specified.

AIC/BIC/HQIC are baselines for deep networks; their regular-model assumptions usually fail because neural networks are singular and non-identifiable.

Algorithms II: Bayesian coding and marginal evidence

Variational approximation

For prior $p(\theta)$ and variational approximation $q_\phi(\theta)$,

$$\mathcal{L}_{\text{var}} = \mathbb{E}_{q_\phi}[-\log p(D | \theta)] + \text{KL}(q_\phi(\theta) \| p(\theta)) = -\text{ELBO} \geq -\log p(D).$$

The coding interpretation of the KL term requires an explicit scheme such as bits-back coding.

KFAC–Laplace approximation

At the MAP estimate $\hat{\theta}$, let $H = \nabla_{\theta}^2[-\log p(D, \theta)]_{\hat{\theta}}$ be positive definite. Then

$$-\log p(D) \approx -\log p(D, \hat{\theta}) + \frac{1}{2} \log \det H - \frac{d}{2} \log(2\pi).$$

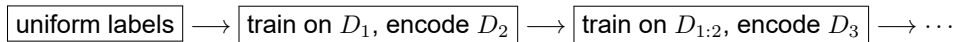
KFAC replaces full curvature by layerwise Kronecker-factored Fisher/GGN blocks.

The estimate depends on the prior, variational family, curvature definition, damping, and the validity of a single-mode Gaussian approximation.

Algorithms III: prequential coding

Ordered data are partitioned into blocks $0 = n_0 < n_1 < \dots < n_m = n$. The first labels use a uniform code; each later block is encoded by a model trained only on previously decoded labels:

$$L_{\text{preq}} = n_1 \log_2 K - \sum_{j=2}^m \sum_{i=n_{j-1}+1}^{n_j} \log_2 p_{\hat{\theta}(D_{<j})}(y_i | x_i).$$



- Inputs x_i are shared side information; only labels are encoded.
- Architecture, optimizer, ordering, block schedule, calibration protocol, and reproducible training procedure must be shared or encoded.
- Implement a blockwise baseline and an online/replay variant; report every training run and wall-clock cost.

Validation: encoder and decoder reproduce the same probabilities without future-label leakage. Shorter per-example length on readily learnable data is a diagnostic, not a theorem.

Algorithms IV: compression-based complexity

A trained network is replaced by a discrete compressed representation C using pruning, quantization, or low-rank factorization. Two related quantities are kept separate:

$$\underbrace{L(C) + L(D \mid C)}_{\text{two-part data code}} \quad \text{and} \quad \underbrace{\ell(C)}_{\text{model description in bits}} .$$

For a prefix-coded hypothesis and bounded loss, a schematic Occam bound is

$$R(C) \leq \hat{R}(C) + \sqrt{\frac{\ell(C) \ln 2 + \ln(1/\delta)}{2n}},$$

with the exact constants determined by the selected theorem.

- The bound applies to the compressed model; relating it to the original model requires a controlled loss or margin distortion.
- The encoder, decoder, precision, sparsity indices, and side information are fixed by the protocol and counted in the code.
- A serialized .pt file size is not a theoretical description length.

Validation: exact round-trip decoding, reproducible bit count, and measured accuracy, NLL, and margin change after compression.

Benchmark protocol and hypotheses

Component	Controlled or varied
Data	synthetic classification, MNIST, Fashion-MNIST; fixed train/validation/test splits
Models	MLP/CNN; width, depth, regularization, multiple random seeds
Training	optimizer, stopping rule, compute budget; MLE-like baseline where required
Outputs	native score, justified nats/bits conversion, components, assumptions, warnings
Resources	wall-clock time, number of training runs, peak memory

Two empirical hypotheses

1. total criterion or code length, normalized per example, versus held-out NLL;
2. consistently normalized penalty or model-description component versus the generalization gap (test NLL minus train NLL, in evaluation mode).

Analysis: Spearman correlation, confidence intervals, rank stability across seeds and sample sizes, and estimator cost relative to model training. Validation data select hyperparameters; the test set is used only for final evaluation.

Compare lower-is-better ranks within each estimator target; unit conversion alone does not create semantic comparability.

Proof-of-concept demonstration

PoC for Technical Meeting 2

1. Train 3–5 MLPs of different widths on one fixed MNIST split.
2. Through the common API, compute parameter count, BIC, two-part MDL, and a blockwise prequential code.
3. Save the configuration, seed, checkpoints, score components, time, and memory.
4. Plot complexity against held-out NLL and the generalization gap.

Basic user workflow

```
result = registry.create("bic", config).estimate(context)
report = BenchmarkRunner(experiment).run(estimators)
```

Final demo output: a comparable result table with native units, rank order, score decomposition, assumptions, warnings, and runtime. One CLI or notebook workflow must reproduce the result from a clean environment.

The PoC validates the interface and the end-to-end data path; it is not yet the final empirical study.

Team responsibilities and algorithm ownership

Member	Algorithm family	Additional responsibilities and deliverable
Alyabushev	AIC, BIC, HQIC, WAIC, WBIC, two-part MDL	Planning; benchmarks and PoC; final comparison tables.
Dementiev	Variational coding; KFAC–Laplace evidence	Planning; library packaging and common API; tests and coverage above 90%.
Kurdyukov	Blockwise and online/replay prequential coding	Planning; public API documentation; reproducible final demonstration.
Chirkov	Compression-based score and generalization bound	Planning; blog post; three-to-five-page technical report.

Cross-review: Alyabushev → Dementiev → Kurdyukov → Chirkov → Alyabushev.

Each algorithm owner supplies the mathematical definition and assumptions, interface implementation, minimal example, unit tests, and documentation section.

Milestones and acceptance criteria

Stage	Verifiable deliverable
Technical Meeting 1	scope, definitions, units, architecture, stack, API, roles, PoC plan
Technical Meeting 2	end-to-end PoC; draft documentation, blog post, and technical report
Checkpoint	installable package; consistent code style; workflows; cross-review
Technical Meeting 3	all estimator families; benchmark; coverage above 90%; demo; final report

Definition of done

- installation and execution succeed in a clean environment;
- every estimator implements one contract and returns a structured result;
- benchmark configurations fix data, training, units, and evaluation protocol;
- tests cover formulas, edge cases, and end-to-end integration;
- documentation states assumptions and provides reproducible examples;
- the report separates established theory, implementation choices, and empirical findings.

Main risks and controls

Risk	Control
Regular-model criteria are applied to singular neural networks	treat them as baselines; state violated assumptions and use a controlled MLE-like run
WAIC/WBIC require posterior inference	use a dedicated adapter; do not report a score when required draws are absent
Laplace is sensitive to prior, curvature, and damping	save the decomposition and run a sensitivity analysis
Prequential coding requires many repeated fits	start with small models; fixed block schedule; online/replay variant
Compression results depend on the exact code and distortion	fix encoder/decoder; count side information; verify round-trip and quality change
Scores use different targets and native scales	retain target and native value; convert only when justified; compare stratified ranks

Resource constraint: first validate one minimal estimator per family on an MLP; add CNNs and the full experiment grid only after the shared interface is stable.

Summary

- `dl_complexity` exposes classical, MDL, Bayesian, prequential, and compression-based estimators through one transparent interface.
- The benchmark relates their native outputs to held-out NLL, the generalization gap, computational cost, and rank stability on controlled models and data.
- The first result is an end-to-end MLP PoC; four algorithm branches then proceed in parallel and undergo cross-review.
- Every result retains its target, assumptions, units, components, and warnings; common software does not erase theoretical differences.

Next technical step: finalize `EstimationContext` and `ComplexityResult`, then implement the BIC, two-part MDL, and prequential PoC.

References

-  H. Akaike (1974), G. Schwarz (1978), and E. Hannan & B. Quinn (1979). Primary definitions of AIC, BIC, and HQIC.
-  S. Watanabe. *Asymptotic Equivalence of Bayes Cross Validation and Widely Applicable Information Criterion in Singular Learning Theory* (JMLR, 2010); *A Widely Applicable Bayesian Information Criterion* (JMLR, 2013).
-  P. Grunwald. *A Tutorial Introduction to the Minimum Description Length Principle*. arXiv:math/0406077, 2004.
-  A. Graves. *Practical Variational Inference for Neural Networks*. NeurIPS, 2011.
-  H. Ritter, A. Botev, D. Barber. *A Scalable Laplace Approximation for Neural Networks*. ICLR, 2018.
-  J. Bornschein, Y. Li, M. Hutter. *Sequential Learning of Neural Networks for Prequential MDL*. arXiv:2210.07931, 2022.
-  S. Arora, R. Ge, B. Neyshabur, Y. Zhang. *Stronger Generalization Bounds for Deep Nets via a Compression Approach*. ICML, 2018.